

BENCHMARK WORKSHOP · JUNE 2026

S3-Compatible Object Storage Benchmark Report

Evaluating MinIO, RustFS, and LibreFS
for Analytical Data Workloads

What this report covers:

- > **Three** benchmark engines: **DuckDB** · **DuckLake** · **Iceberg** -- progressively realistic workloads
- > **206** result files · n=3 runs per configuration · mean +/- stdev throughout
- > **Expert** analysis of GC vs. Rust ownership, erasure-coding quorum, and metadata latency
- > **Critical** production safety findings for stateful streaming workloads (RisingWave Hummock)
- > **Actionable** decision framework for storage system selection

Author: Mojtaba Banaie · linkedin.com/in/smbanaie/

Machine: DELL Precision 5520 · i7-7820HQ · 32 GB RAM · Windows 11 Pro

Date: June 22, 2026 · Python 3.13.6 via uv · Docker Desktop

Repository: github.com/sepahram-school/workshops

Table of Contents

Table of Contents	2
1. Executive Summary	4
Headline Results (32 MB Workloads)	4
2. Motivation: Why This Benchmark Exists	4
3. Why Simple Read/Write Tests Are Not Enough	5
3.1 Concurrent Workloads Expose GC and Quorum Latency	5
3.2 Small Files Expose Per-Object Metadata Overhead	5
3.3 Analytical Engines Use Columnar, Not Sequential Access	5
3.4 Lakehouse Metadata Creates Transactional Dependencies	5
4. Benchmark Design: A Three-Engine Strategy	5
4.1 Engine 1 -- DuckDB Direct (benchmark_duckdb.py)	6
4.2 Engine 2 -- DuckLake (benchmark_ducklake.py)	6
4.3 Engine 3 -- Iceberg-Style (benchmark_iceberg.py)	6
5. Test Environment	7
6. Systems Under Test	7
7. Docker Cluster Architecture	8
8. Benchmark Results	8
8.1 DuckDB Engine -- Columnar Parquet I/O	8
Write Throughput (MB/s, 20 threads)	8
Read Throughput (MB/s)	8
Heavy Mode -- Concurrent Read + Write	9
8.2 DuckLake Engine -- Lakehouse Metadata Performance	9
Write Throughput (MB/s)	9
Read, Heavy, and Transaction Results	9
8.3 Iceberg Engine -- File-Pattern Performance	9
Write and Read Latency P50 (seconds, lower is better)	9
Metadata Operations P50 (seconds, lower is better)	10
8.4 Cross-Engine Summary (1 MB, P50)	10
8.5 Winner Scorecard	10
9. Key Findings & Expert Analysis	11
Finding 1: RustFS Performance Lead Is Architecturally Explained	11
Finding 2: Small-File vs Large-File Split Maps to Production Archetypes	11
Finding 3: MinIO Aggregate Heavy-Mode Throughput Is Underrated	11
Finding 4: DuckLake Is Production-Ready on All Three Backends	11
Finding 5: LibreFS High Variance Is the Most Underappreciated Signal	11
Finding 6: Benchmarks Cannot Capture Consistency Failure Modes	12
10. Production Observations	12
10.1 RustFS: Critical Failures as RisingWave Hummock Backend	12

10.2 LibreFS: Production Validated for Streaming Workloads	12
11. Recommendations.....	13
Decision Framework	13
12. Conclusion	14
Appendix A -- Methodology Peer Review	15
P0 Critical Fixes	15
P1 Major Fixes	16
Appendix B -- Known Limitations.....	16
Appendix C -- How to Reproduce.....	17
Appendix D -- DuckDB Benchmark Methodology	17
Classic boto3 vs DuckDB: Two Different Access Patterns	17
Why DuckDB Over Alternatives?	17
Test Data Schema	18
Workload Modes	18
write-only	18
read-only.....	18
mixed	18
heavy	18
Metrics Collected.....	19
Interpreting DuckDB Results.....	19
Appendix E -- DuckLake Benchmark Methodology	19
Architecture	19
Table Schema	20
Data Generation.....	20
All 8 Benchmark Modes	20
Why concurrent Mode Uses Per-Worker Metadata	20
Run Isolation and Cleanup.....	21
Interpreting DuckLake Results.....	21
Appendix F -- Iceberg Benchmark Methodology	21
How Iceberg Metadata Works on S3	21
Why DuckDB for Iceberg Simulation (Not Pylceberg)?	22
Table Schema	22
Data Generation.....	22
All 9 Benchmark Modes	22
Iceberg vs DuckLake Comparison	23
Metrics Collected.....	23
What Iceberg Tests That Other Engines Do Not	24
Interpreting Iceberg Results	24

1. Executive Summary

This report presents a rigorous, reproducible benchmark comparing three S3-compatible object storage systems - **MinIO**, **RustFS**, and **LibreFS** - under realistic analytical data workloads spanning 206 result files across three independent benchmark engines, with methodology validated through a senior data architect peer review.

The central finding is a fundamental tension between benchmark performance and production reliability. RustFS dominates **7 out of 10** benchmark categories and delivers **2-8x lower latency** than MinIO on Iceberg workloads. Yet the same system caused critical, unrecoverable SSTable manifest corruption in two independent RisingWave Hummock deployments.

Headline Results (32 MB Workloads)

Dimension	Winner	Margin
DuckDB large-file writes	LibreFS	28.8 MB/s vs MinIO 20.2 MB/s (+42%)
DuckDB read throughput	RustFS	61.0 MB/s vs MinIO 50.7 MB/s (+20%)
DuckDB concurrent heavy	MinIO	29.0 MB/s highest aggregate throughput
Iceberg write latency	RustFS	0.059s vs MinIO 0.121s (2x faster)
Iceberg reads/snapshots/TT	RustFS	2-8x lower P50 latency across the board
DuckLake ACID transactions	RustFS	36ms vs MinIO 54ms P50
DuckLake time-travel	RustFS	0.228s vs MinIO 0.388s P50
Production reliability	MinIO/LibreFS	RustFS was not stable under Hummock compaction

[!] CRITICAL: RustFS must NOT be used as a RisingWave Hummock backend or stateful streaming storage yet. MinIO and LibreFS are both validated safe for streaming workloads. For pure analytical batch pipelines, RustFS leads on performance.

2. Motivation: Why This Benchmark Exists

MinIO became the de facto standard for self-hosted S3-compatible storage. In 2025, MinIO moved WebUI, LDAP, and distributed erasure coding at scale behind the proprietary **AIStor** product, while the core AGPLv3 server remained open source. This created a real licensing question: is there a viable alternative?

- **MinIO / MinIO AIStor** – The benchmark baseline, Core product now offered under three subscription tiers: **AIStor Free** (single-node, no-cost, no redistribution/resale), **AIStor Enterprise Lite** (distributed, up to 400TiB, self-support), and **AIStor Enterprise** (full-scale, premium support). The source code remains under **AGPLv3**, but official builds are no longer freely provided. **Not safe for commercial embedding** without a commercial license or building from source and complying with AGPLv3.
- **RustFS** - MinIO-compatible rewrite in Rust; Apache 2.0 licensing safe for commercial embedding
- **LibreFS** - Community fork of MinIO preserving full open-source features under AGPLv3

3. Why Simple Read/Write Tests Are Not Enough

Most S3 benchmarks measure sequential PUT/GET throughput - useful for raw network saturation, but almost nothing about production behavior.

3.1 Concurrent Workloads Expose GC and Quorum Latency

A data pipeline does not wait for one write to finish. Kafka consumers, BI tools, and compaction jobs all overlap. Under concurrent load, Go-based systems experience GC-induced tail latency; MinIO and LibreFS also accumulate erasure-coding quorum latency.

-> Evidence: MinIO write P50 jumps from ~2s (sequential) to 9.0s (heavy mode) - expected 3-node quorum behavior, not a failure. Aggregate throughput remains highest at 29.0 MB/s.

3.2 Small Files Expose Per-Object Metadata Overhead

When lakehouses write thousands of small Parquet files (1-5 MB each) -- the Iceberg streaming pattern -- the metadata plane becomes the bottleneck, not the data plane.

-> Evidence: RustFS is 2x faster than MinIO at 1 MB DuckDB writes (13.2 vs 6.6 MB/s) but only 8% faster at 32 MB. At 1 MB Iceberg writes: 3.5x faster (0.116s vs 0.412s P50).

3.3 Analytical Engines Use Columnar, Not Sequential Access

DuckDB, Trino, and Spark use **columnar predicate pushdown** -- reading specific row groups via ranged HTTP GET requests, not full file downloads. This rewards low per-request latency over raw bandwidth.

3.4 Lakehouse Metadata Creates Transactional Dependencies

Every Iceberg/DuckLake table commit writes manifest files, snapshot pointers, and transaction logs as S3 objects. S3 consistency semantics determine whether the lakehouse is reliable or subtly corrupt.

-> Evidence: All three systems process 200 ACID transactions at 36-54ms P50 with 100% success -- DuckLake is production-ready. But MinIO DuckLake heavy write P50 jumps to 9.075s, confirming quorum latency persists across the full lakehouse write path.

4. Benchmark Design: A Three-Engine Strategy

A fourth engine -- **Classic boto3** (benchmark.py, raw put_object/get_object) -- also exists in the workshop repository as a baseline reference but is not covered in this report. See the workshop README for its results.

Engine	Tests	Metadata	ACID	Real-World Analog
DuckDB	Analytical Parquet I/O	None	No	Data lake queries, OLAP
DuckLake	Lakehouse + SQLite metadata	SQLite	Yes	DuckLake / Delta Lake
Iceberg	Iceberg file-based metadata on S3	S3 files	Yes	Apache Iceberg / prod data lakes

4.1 Engine 1 -- DuckDB Direct (benchmark_duckdb.py)

What it measures: Raw columnar read/write throughput via DuckDB httpfs extension. Bypasses all lakehouse metadata layers; directly exercises the object store core PUT/GET performance under realistic OLAP conditions.

Why DuckDB over alternatives? pandas+s3fs loads entire files into memory; Spark requires a JVM cluster; Trino needs cluster setup. DuckDB provides zero-config S3 support, Parquet-native columnar streaming, single binary.

Design Aspect	Approach	Rationale
Write method	COPY (SELECT * FROM read_parquet(...)) TO s3://	No intermediate table; columnar serialization end-to-end
Read queries	SELECT SUM(value), COUNT(*), MAX(timestamp)	Forces full page reads -- not just Parquet footer (Appendix A)
Extension loading	LOAD httpfs once at adapter construction	Eliminates extension setup from timing
Thread model	Per-thread connection; SET threads=1	Prevents Python x DuckDB thread explosion
Warmup	10% of files at target thread count	Fills pools; warms page cache
Heavy mode	N/2 writers + N/2 readers; re-list keys every 5s	Exposes read/write contention; readers see fresh writes

4.2 Engine 2 -- DuckLake (benchmark_ducklake.py)

What it measures: Lakehouse transactional metadata performance. DuckLake layers a **SQLite-backed metadata catalog** on top of DuckDB + S3, providing ACID transaction semantics, snapshot isolation, and time-travel.

Why SQLite metadata (not on S3)? All three targets use the same local SQLite instance -- isolating S3 performance as the only variable. Schema: (id UUID, ts TIMESTAMP, sensor_id INT, value DOUBLE, label VARCHAR).

Mode	What It Tests
write-only	Bulk INSERT throughput through full DuckLake write pipeline
read-only	Analytical query latency against seeded lakehouse tables
heavy	Concurrent reads + writes for 30 seconds
transactions	200 multi-table ACID: BEGIN -> INSERT orders + items -> COMMIT
time-travel	Snapshot creation + AT VERSION n historical queries
snapshots	Snapshot metadata listing overhead
concurrent	Multiple independent DuckLake writers (per-worker SQLite)

4.3 Engine 3 -- Iceberg-Style (benchmark_iceberg.py)

What it measures: File-pattern simulation of Apache Iceberg -- multiple small Parquet data files, manifest listing, snapshot reads, time travel, and change detection -- without requiring a full catalog runtime.

Each batch writes `data/events/000001-<uuid>.parquet` (mirroring Iceberg data file naming). Manifest listing simulated via `glob(s3://bucket/data/**/*.*.parquet)` -- testing S3 LIST + prefix scan performance critical for Iceberg manifest resolution. PyIceberg excluded due to known PyArrow multipart upload incompatibilities.

Mode	What It Tests
write-only	Parquet file creation throughput at Iceberg granularity
read-only	Analytical query latency on individual data files
heavy	Concurrent writers + readers for 30 seconds
time-travel	Write batches to distinct paths; read each historical path
snapshots	File listing + catalog metadata overhead
change-detection	Diff two snapshot paths to detect new/removed files
update	Write new Parquet representing an updated partition
delete	Write new Parquet representing a delete marker

5. Test Environment

Specification	Value
Machine	DELL Precision 5520
CPU	Intel Core i7-7820HQ @ 2.90 GHz
RAM	32.0 GB (31.9 GB usable)
OS	Windows 11 Pro (64-bit)
Docker	Docker Desktop
Python	3.13.6 via uv
Benchmark Date	June 22, 2026
Runs per config	3 (n=3) -- mean +/- stdev reported

[i] All benchmarks ran in isolation -- one cluster active at a time, full bucket cleanup between runs. All traffic traverses Docker loopback (127.0.0.1). Real multi-host deployments will show different absolute numbers; relative ordering expected to hold.

6. Systems Under Test

System	Founded	Language	License	Key Differentiator
MinIO	2015	Go	AGPLv3	Reference S3 implementation; most production deployments
RustFS	2024	Rust	Apache 2.0	MinIO-compatible rewrite; memory safety + permissive license
LibreFS	2025	Go	AGPLv3	Community fork preserving WebUI, LDAP, erasure coding

MinIO (2015, AGPLv3) is the de facto standard. In 2025, advanced features moved to AIStor (proprietary). Core AGPLv3 server remains open source. 3-node distributed erasure-coded cluster tested here.

RustFS (2024, Apache 2.0) is a ground-up Rust rewrite of the MinIO-compatible S3 API. No garbage collector; memory managed via Rust ownership rules. Apache 2.0 license safe for commercial embedding. Young project as of mid-2026 -- with, as documented here, at least one serious production stability issue.

LibreFS (2025, AGPLv3) is a community fork of MinIO pre-AIStor, preserving WebUI, LDAP/OIDC, and distributed erasure coding. Architecturally identical to MinIO; same Go runtime and erasure-coding behavior.

7. Docker Cluster Architecture

System	Endpoint	Auth
RustFS	http://127.0.0.1:9000	rustfsadmin / rustfsadmin
LibreFS	http://localhost:9100	admin / password123
MinIO	http://localhost:9200	minioadmin / minioadmin123

All three systems run as 3-node Docker clusters on the same host. MinIO and LibreFS use a distributed erasure-coded configuration -- identical architecture, different codebases. Both require **write quorum across all 3 nodes** before returning success. This is the primary source of write latency under concurrent load -- the critical factor for interpreting heavy-mode results in Section 8.

8. Benchmark Results

Results represent n=3 runs per configuration. Throughput = mean +/- stdev (MB/s). Latency = P50 in seconds (lower is better). All configurations achieved 100% success rates. **Bold** = winner per row.

8.1 DuckDB Engine -- Columnar Parquet I/O

Write Throughput (MB/s, 20 threads)

Target	1 MB Files	32 MB Files
MinIO	6.6 +/- 0.7	20.2 +/- 1.6
RustFS	13.2 +/- 1.5	21.8 +/- 1.0
LibreFS	6.7 +/- 0.7	28.8 +/- 3.3

-> Leader switches by file size. RustFS 2x faster at 1 MB (GC-free Rust vs Go). LibreFS leads 32 MB at 28.8 MB/s (+42% over MinIO). Note high stdev on LibreFS 32 MB (+/-3.3) -- risk signal for sustained production load.

Read Throughput (MB/s)

Target	1 MB Files	32 MB Files
MinIO	14.3 +/- 3.5	50.7 +/- 4.4
RustFS	17.0 +/- 1.2	61.0 +/- 7.1
LibreFS	9.7 +/- 2.0	54.5 +/- 10.8

[!] RustFS leads both sizes with lower variance (+/-1.2 at 1 MB vs MinIO +/-3.5). LibreFS +/-10.8 variance at 32 MB is the highest single metric in this entire report -- a material tail-latency risk for SLA-sensitive query workloads.

Heavy Mode -- Concurrent Read + Write

Target	Write P50	Reads Completed	Aggregate Throughput
MinIO	9.0s	10	29.0 MB/s
RustFS	0.77s	0	(timing artifact -- see note)
LibreFS	1.62s	15	balanced

[j] MinIO 9.0s write P50 is NOT a failure -- it is expected 3-node erasure-coding quorum behavior under concurrent load. MinIO achieves the highest aggregate throughput (29.0 MB/s). RustFS 0 reads is a timing artifact: readers issued LIST requests before writers populated keys.

8.2 DuckLake Engine -- Lakehouse Metadata Performance

Write Throughput (MB/s)

Target	1 MB	32 MB
MinIO	0.6 +/- 0.1	19.7 +/- 1.0
RustFS	0.4 +/- 0.1	19.9 +/- 0.4
LibreFS	0.4 +/- 0.2	11.3 +/- 3.0

-> At 32 MB, MinIO and RustFS effectively tied -- DuckLake SQLite overhead normalizes storage differences. LibreFS underperforms (11.3 MB/s, +/-3.0 variance), suggesting interaction issues with DuckLake metadata commit pattern.

Read, Heavy, and Transaction Results

Target	Read P50 1MB	Read P50 32MB	Heavy Write P50	Heavy Read P50	Txn P50	Time-Travel P50
MinIO	0.050s	0.055s	9.075s	0.026s	0.054s	0.388s
RustFS	0.040s	0.059s	3.215s	0.029s	0.036s	0.228s
LibreFS	0.030s	0.090s	3.239s	0.062s	0.041s	0.284s

[ok] 100% ACID transaction success across all targets at 36-54ms P50 validates DuckLake as production-ready on all three backends. All three systems also maintain sub-100ms read P50 even under heavy concurrent write pressure.

8.3 Iceberg Engine -- File-Pattern Performance

Write and Read Latency P50 (seconds, lower is better)

Target	Write 1MB	Write 32MB	Read 1MB	Read 32MB
MinIO	0.412s	0.121s	0.024s	0.023s
RustFS	0.116s	0.059s	0.029s	0.014s
LibreFS	0.228s	0.172s	0.066s	0.034s

-> RustFS 3.5x faster than MinIO at 1 MB writes (Iceberg streaming pattern) and 2x faster at 32 MB. LibreFS is SLOWER than MinIO at 32 MB Iceberg writes -- reversal from its DuckDB large-file lead. Not suited to Iceberg many-small-files pattern.

Metadata Operations P50 (seconds, lower is better)

Operation	MinIO	RustFS	LibreFS
Heavy write	0.819s	0.257s	1.098s
Heavy read	0.112s	0.082s	0.072s
Time-travel 1 MB	0.171s	0.054s	0.114s
Time-travel 32 MB	0.128s	0.049s	0.142s
Snapshots 1 MB	0.108s	0.071s	0.183s
Snapshots 32 MB	0.168s	0.049s	0.116s
Change-detection 1 MB	0.505s	0.060s	0.167s
Change-detection 32 MB	0.100s	0.053s	0.202s
Update	0.130s	0.144s	0.299s
Delete	0.463s	0.081s	0.186s

-> RustFS dominates all metadata operations by 2-8x. Change-detection at 1 MB: 8x faster than MinIO (0.060s vs 0.505s) -- reflects superior S3 LIST latency, critical for Iceberg manifest resolution. Update is the only metric where MinIO leads (10% margin).

8.4 Cross-Engine Summary (1 MB, P50)

Target	Mode	DuckDB	DuckLake	Iceberg
MinIO	write	2.335s	0.248s	0.412s
RustFS	write	1.127s	0.425s	0.116s
LibreFS	write	2.306s	0.475s	0.228s
MinIO	read	0.786s	0.050s	0.024s
RustFS	read	0.556s	0.040s	0.029s
LibreFS	read	0.864s	0.030s	0.066s
MinIO	heavy	2.475s	9.075s	0.819s
RustFS	heavy	1.495s	3.215s	0.257s
LibreFS	heavy	3.052s	3.239s	1.098s

8.5 Winner Scorecard

Target	Category Wins	Strongest Domain
RustFS	7 / 10	Iceberg workloads, DuckDB reads, DuckLake transactions, concurrent loads
LibreFS	2 / 10	DuckDB large-file writes, DuckLake small reads
MinIO	1 / 10	DuckDB heavy-mode aggregate throughput

9. Key Findings & Expert Analysis

Finding 1: RustFS Performance Lead Is Architecturally Explained

Garbage collection. Go (MinIO, LibreFS) uses a concurrent tracing GC with periodic stop-the-world pauses. Under concurrent load with many short-lived request objects, GC pressure increases. Rust has no GC -- memory management is resolved at compile time via ownership rules. The 9.0s vs 0.77s write P50 in DuckDB heavy mode is not a 12x better algorithm; it is deterministic memory management vs GC-induced tail latency under sustained load.

Erase coding quorum. MinIO and LibreFS require write quorum across 3 nodes before returning success. Under concurrent write load, each operation waits for the slowest node. RustFS cluster configuration may not enforce equivalent write durability -- making the latency comparison partially asymmetric on the durability axis.

[i] If RustFS is configured to enforce the same 3-node erasure-coded write durability as MinIO, some portion of its latency advantage will narrow. The benchmark compares systems in their default cluster configurations.

Finding 2: Small-File vs Large-File Split Maps to Production Archetypes

- **High-cardinality small-file workloads** (streaming ingestion, Kafka->S3, Iceberg streaming tables) -- favor **RustFS**. Per-object overhead dominates at small sizes.
- **Large-file batch workloads** (Spark writes, dbt exports, nightly compaction) -- favor **LibreFS** (28.8 MB/s) or **RustFS** (21.8 MB/s). LibreFS variance (+/-3.3) must be profiled at production concurrency before adoption.

Finding 3: MinIO Aggregate Heavy-Mode Throughput Is Underrated

MinIO pipelines and batches writes internally, achieving the highest total data movement (29.0 MB/s aggregate in DuckDB heavy mode) at the cost of per-operation latency. For bulk ingestion, backup, and large-scale compaction -- workloads where total data moved matters more than individual operation speed -- MinIO model may be superior.

Finding 4: DuckLake Is Production-Ready on All Three Backends

100% ACID transaction success across 200 multi-table transactions at 36-54ms P50 validates DuckLake as a viable production lakehouse format. Teams can adopt DuckLake with confidence on any of these storage backends.

Finding 5: LibreFS High Variance Is the Most Underappreciated Signal

LibreFS shows the highest result variance: +/-3.3 MB/s on 32 MB DuckDB writes, +/-10.8 MB/s on 32 MB reads. High variance typically indicates background GC cycles, internal write lock contention, or buffering strategies creating periodic latency spikes.

[!] For SLA-sensitive workloads, unpredictable latency is often more damaging than consistently higher latency. A query engine with a 60-second timeout that encounters a 65-second tail-latency spike fails regardless of its P50. Profile LibreFS P99 at production concurrency before committing.

Finding 6: Benchmarks Cannot Capture Consistency Failure Modes

All three systems achieved 100% success rates across all benchmark configurations. Yet RustFS caused unrecoverable data corruption in two production RisingWave Hummock deployments -- while MinIO and LibreFS both remained stable under the identical workload. **Benchmarks measure throughput and latency -- they do not measure correctness under adversarial concurrent access patterns with strict ordering invariants.** The contrast here is instructive: MinIO and LibreFS share a quorum-based consistency model that happens to satisfy Hummock's ordering invariants; RustFS does not. Benchmark success is necessary but not sufficient for production adoption, especially for systems with younger codebases.

10. Production Observations

10.1 RustFS: Critical Failures as RisingWave Hummock Backend

[!] [CRITICAL] DATA CORRUPTION -- DO NOT USE RUSTFS AS A RISINGWAVE HUMMOCK BACKEND

RustFS was evaluated as a backend for **RisingWave Hummock** -- the LSM-tree-based object storage layer that RisingWave uses for streaming state persistence. In **two independent deployments**, RustFS caused catastrophic failures requiring full system recovery with data loss. **MinIO and LibreFS were both tested and remained stable under identical Hummock workloads.**

Failure mode: Hummock compaction requires each SSTable to be fully written and durable before its reference appears in the manifest. RustFS apparently does not satisfy this strict ordering guarantee consistently -- the manifest was updated before referenced SSTable data was fully readable, corrupting the LSM-tree state beyond recovery without a full data wipe.

Why the benchmark did not catch this: The heavy mode tests concurrent reads and writes but does not exercise Hummock's specific invariants: atomic multi-object commits, strict read-after-write consistency within a single client, and deterministic listing order. RustFS satisfies loose S3 API contracts -- sufficient for benchmark success checks -- while violating the stronger consistency guarantees Hummock compaction relies on.

[ok] Resolution: Switching the Hummock backend from RustFS to MinIO immediately resolved both failures. LibreFS was subsequently validated as stable under the same Hummock workload. Both MinIO and LibreFS are safe choices for RisingWave deployments.

10.2 LibreFS: Production Validated for Streaming Workloads

LibreFS was validated as stable under RisingWave Hummock compaction workloads -- the same workload that caused RustFS failures. This is a meaningful production finding: LibreFS's architectural inheritance from MinIO (same Go runtime, same erasure-coding quorum model, same read-after-write consistency guarantees) carries over to its consistency behavior under strict ordering requirements.

Operationally, LibreFS inherits MinIO's full administrative tooling. The preserved WebUI, LDAP, and OIDC features work as expected. Primary operational risk remains its smaller maintenance community -- security patches depend on fork maintainers keeping pace with upstream MinIO. Teams adopting LibreFS for production streaming workloads should actively monitor the upstream patch cadence.

11. Recommendations

Use Case	Recommended	Rationale
Apache Iceberg data lake	RustFS	2-8x lower latency across all Iceberg modes; fastest snapshots, time-travel, change-detection
DuckDB / Trino / Spark OLAP reads	RustFS	Highest read throughput (61 MB/s at 32 MB), lowest variance (+/-1.2 at 1 MB)
Large Parquet batch writes (>32 MB)	LibreFS	28.8 MB/s (+42% over MinIO); validate P99 variance before committing
Streaming ingestion (<5 MB files)	RustFS	2x MinIO write throughput on 1 MB files; lowest per-object overhead
DuckLake ACID lakehouse	RustFS	36ms transaction P50, 100% ACID success, fastest time-travel (0.228s)
High-concurrency mixed loads	MinIO	Highest aggregate heavy-mode throughput (29 MB/s); proven 3-node quorum model
RisingWave Hummock backend	MinIO or LibreFS	Both validated stable under Hummock compaction; RustFS caused SSTable manifest corruption
ClickHouse Separate Storage from Compute	MinIO or LibreFS	Both provide the strong read-after-write consistency ClickHouse S3 storage requires; LibreFS adds WebUI + monitoring
Need WebUI + LDAP + AGPLv3 without AIStor	LibreFS	Drop-in MinIO replacement with preserved open-source features; monitor P99 variance
Apache 2.0 permissive commercial license	RustFS	No copyleft obligations; safe for embedding in commercial analytical pipelines

Decision Framework

RisingWave Hummock backend (stateful streaming)?

- **MinIO or LibreFS** -- both validated stable under Hummock compaction. Do NOT use RustFS; it caused SSTable manifest corruption in two independent deployments.

ClickHouse Separate Storage from Compute (S3-backed MergeTree)?

- **MinIO or LibreFS** -- ClickHouse S3 storage requires strong read-after-write consistency for part merges and mutations. Both MinIO and LibreFS provide this via their erasure-coded quorum model. LibreFS adds the advantage of a built-in WebUI for bucket monitoring and a familiar MinIO API. RustFS is not recommended until its consistency guarantees are formally verified for write-heavy merge workloads.

Primary file size < 5 MB? (streaming ingestion, frequent Iceberg commits, Kafka->S3 micro-batches)

- **RustFS** -- 2-3.5x faster writes, 3x faster Iceberg ops; per-object overhead is lowest

Primary file size > 32 MB? (batch Spark writes, dbt exports, nightly compaction)

- **LibreFS** (28.8 MB/s, +42% over MinIO) -- but profile P99 variance under sustained load first

High-concurrency sustained throughput is the primary concern?

- **MinIO** -- highest aggregate heavy-mode throughput (29.0 MB/s); proven 3-node quorum model optimized for bulk pipelining

ACID lakehouse with frequent transactions or time-travel queries?

- -> **RustFS** -- fastest DuckLake transactions (36ms P50), fastest time-travel (0.228s P50)

Need WebUI + LDAP + AGPLv3 without paying for AIStor?

- -> **LibreFS** -- MinIO fork with all preserved open-source features; production-validated for streaming workloads

12. Conclusion

RustFS is the fastest system in this benchmark -- winning 7 of 10 category comparisons, delivering 2-8x lower Iceberg latency, and showing the most consistent per-operation response times. For pure analytical batch workloads -- Iceberg pipelines, DuckDB data lakes, DuckLake lakehouses -- RustFS delivers a compelling combination of performance and Apache 2.0 licensing.

But benchmark performance is not production readiness. The RisingWave Hummock failures reveal that RustFS's consistency model has untested edges that cause data corruption under high-frequency concurrent access with strict ordering invariants. **MinIO and LibreFS were both validated stable** under identical Hummock compaction workloads, confirming that their shared Go runtime and erasure-coding quorum model provides the consistency guarantees streaming databases require. Do not use RustFS for stateful streaming until its consistency guarantees are formally characterized and independently verified.

MinIO remains the proven default for production deployments -- the most production hours, the best-documented consistency model, and the largest community. Its aggregate heavy-mode throughput advantage reveals it is specifically engineered for sustained concurrent loads at scale.

LibreFS is stronger than its benchmark variance suggests. Its large-file write lead (28.8 vs 20.2 MB/s) is genuine, its RisingWave production validation is meaningful, and for teams that need WebUI, LDAP, and AGPLv3 without AIStor it is the clearest choice. The high benchmark variance deserves monitoring but should not overshadow a solid production track record under the tested streaming workload.

[i] Treat benchmark results as evidence for a hypothesis; treat production observation as the verdict. This report required 3 engines, 11 peer-reviewed methodology fixes, 206 result files, and two real production failures before a complete picture emerged.

Appendix A -- Methodology Peer Review

The benchmark scripts underwent a structured senior data architect peer review. All P0 critical fixes were applied before the reported results were collected. Each fix includes the rationale for why it is viable for production benchmarking.

P0 Critical Fixes

#	Issue	Fix Applied	Why Viable
1	SELECT COUNT(*) reads Parquet footer only -- all read results measured metadata, not storage I/O	Replaced with SELECT SUM(value), COUNT(*), MAX(timestamp) -- forces DuckDB to materialize data pages	Column-materializing queries are exactly what OLAP workloads do; this tests real analytical I/O
2	Different thread counts per target (SeaweedFS:5 vs others:20) -- 4x mechanical bias	--thread-sweep flag runs at 1,5,10,20 threads for all targets; results show scaling curves	Letting the data show where each system saturates is more honest than presetting limits
3	New DuckDB connection per operation (100-500ms per file) -- dominated small-file latency	DuckDBAdapter class: extensions loaded ONCE at construction, connection reused per worker	Matches production: connection pools are standard in data pipeline frameworks
4	Heavy-mode readers use stale key list -- reads hit cached files, not fresh writes	Re-list keys every 5 seconds in DuckDB; DuckLake reads from table directly (includes fresh inserts)	Periodic re-listing simulates real analytical workloads that discover new data incrementally
5	INSTALL httpfs attempts network download from DuckDB extension repo during benchmark	Changed to LOAD httpfs -- httpfs is built-in, no download needed	Eliminates network dependency; makes benchmarks reproducible offline
6	DuckDB CPU thread explosion: 20 Python threads x 16 DuckDB threads = 320 threads on 16 cores	SET threads=1 per connection -- limits DuckDB internal parallelism	Without this, benchmarks measure OS context-switching overhead, not S3 performance
7	run_write_phase O(n ²): creates boto3 client + lists ALL objects inside every batch loop	Moved to single boto3 list_objects_v2 AFTER all writes complete	Reduces write phase from O(n ²) to O(n); measures actual S3 bytes once
8	run_write_phase size_mb was cumulative total -- each batch showed total, not batch size	Backfill all results with total S3 size after measurement; throughput = total/total_time	Aggregate throughput (total data / total time) is the correct metric for sequential writes
9	run_read_phase queried duckdb_parquet_metadata(my_db.events) -- fails on DuckLake tables	Replaced with boto3 list_objects_v2 measurement ONCE before queries	DuckLake tables do not expose duckdb_parquet_metadata; boto3 gives actual S3 object sizes
10	run_heavy_phase used fabricated 100 bytes/row for size estimates	Set size_mb=0 and throughput_mbps=0 -- heavy mode reports counts + latency, not throughput	Heavy mode value is contention measurement (latency, success rate), not throughput
11	w_metrics[mode] = heavy_write crashes when no writes succeeded (NoneType)	Guard with if w_metrics: before accessing	Prevents crash on 100% write failure scenarios

P1 Major Fixes

#	Issue	Fix Applied	Why Viable
12	Warmup too small (3 files at 5 threads) -- TCP slow-start, connection pooling not saturated	Changed to 10% of files at target thread count	Ensures connection pools are warm; mirrors production steady-state
13	Inefficient write (CREATE TABLE _bench_tmp intermediate)	Changed to COPY (SELECT * FROM read_parquet(...)) TO s3:// -- single operation	Eliminates unnecessary local I/O; measures S3 write path only
14	DuckLake SQLite-on-local-disk modes (transactions, time-travel, snapshots, concurrent) do not test S3	Documented: these modes test DuckLake metadata overhead, not storage	Metadata overhead is a real cost of lakehouse guarantees; measuring it is valid, just not for S3 comparison
15	DuckLake run_read_phase queried duckdb_parquet_metadata on DuckLake table	Replaced with boto3 list_objects_v2 for actual S3 size	DuckLake manages Parquet files internally; boto3 gives ground truth
16	No --thread-sweep in DuckLake benchmark	Added --thread-sweep flag (1,5,10,20 threads)	Enables scaling curve analysis for all targets
17	Dead code: get_s3_object_size(), get_parquet_file_size_s3() never called	Removed both functions	Reduces code surface; avoids confusion about which sizing method is used
18	Dead code: time.perf_counter() - time.perf_counter() always ~0 then overwritten	Removed the dead line	Cleans up misleading code
19	read-only mode writes first (misleading name)	read-only seeds data in separate untimed step, then measures reads only	Fair: read performance measured on pre-existing data, not mixed with write latency

Appendix B -- Known Limitations

Limitation	Impact	Mitigation
Single-host Docker loopback	No real network latency; no horizontal scaling	Documented; results are loopback throughput; relative ordering expected to hold on real networks
OS page cache on reads	Cached reads are faster than cold reads	Documented; drop_caches not supported on Windows
Parquet encoding overhead conflated with storage	Write throughput includes CPU encoding time	Relative ranking preserved; absolute numbers are end-to-end
No S3 ListObjects benchmark	Missing metadata-heavy workload	Planned for next iteration; Iceberg snapshot listing partially covers this
No system-level telemetry	Cannot explain why one system is slower	Planned for next iteration: per-target container CPU/memory monitoring
Incompressible payloads not enforced	Throughput numbers may compress better/worse across targets	Documented; relative comparison valid when payload is identical
SeaweedFS excluded	Fourth viable alternative not measured	Workshop #9 dedicated SeaweedFS benchmark

Limitation	Impact	Mitigation
Durability asymmetry	3-node erasure MinIO/LibreFS vs RustFS default cluster configuration	Documented in Section 9, Finding 1
DuckLake SQLite modes not S3 comparisons	Transactions/TT/snapshots measure metadata overhead, not storage	Excluded from cross-target S3 comparison tables

Appendix C -- How to Reproduce

```
git clone https://github.com/sepahram-school/workshops && cd workshops/workshop-08
uv run python generate_data_parquet.py --size all --files 100
docker compose -f docker-compose-minio.yml up -d
uv run python benchmark_duckdb.py --target all --mode mixed --sizes 1mb,32mb --runs 3
uv run python benchmark_ducklake.py --target all --mode heavy --sizes 1mb,32mb --runs 3
uv run python benchmark_iceberg.py --target all --mode mixed --sizes 1mb,32mb --runs 3
```

Results saved to results_duckdb/, results_ducklake/, results_iceberg/ -- organized by mode, size, and target.

[ok] 206 result files: 41 DuckDB | 48 DuckLake | 117 Iceberg. All benchmarks fully reproducible.

Appendix D -- DuckDB Benchmark Methodology

Classic boto3 vs DuckDB: Two Different Access Patterns

Aspect	Classic (benchmark.py)	DuckDB (benchmark_duckdb.py)
Client	boto3 (AWS SDK)	DuckDB with httpfs extension
Data format	Raw binary (.bin)	Parquet (.parquet)
Write operation	put_object() -- raw byte stream	COPY TO s3:// -- columnar write
Read operation	get_object() -- raw byte stream	read_parquet() -- columnar scan
What it tests	Raw S3 throughput	Analytical workload on S3
Real-world use	File transfers, backups, blob storage	Data lakes, OLAP queries, analytics

Why not just boto3? Four reasons modern data engineering workloads differ from raw byte transfers:

- **Columnar access** -- analytics queries read specific columns (SELECT category, AVG(value)), not entire files; irrelevant columns are never fetched from S3
- **Predicate pushdown** -- WHERE clauses skip irrelevant Parquet row groups entirely; DuckDB communicates filter predicates to the storage layer
- **Compression** -- Parquet files transfer compressed bytes over the wire; boto3 raw benchmarks transfer uncompressed byte arrays
- **Multipart uploads** -- DuckDB uses S3 multipart upload protocol differently than boto3, with different chunk sizes and concurrency patterns

Why DuckDB Over Alternatives?

DuckDB was chosen as the analytical S3 client because it represents how real data lake query engines access object storage -- not as simple byte streams, but as typed columnar scans with predicate pushdown.

Alternative	Why DuckDB is Superior
pandas + s3fs	pandas loads entire files into memory; DuckDB streams and processes in chunks
sqlite	No native S3 support; requires local file copies
Spark	Heavy JVM dependency; overkill for benchmarking the storage layer in isolation
Presto / Trino	Requires cluster setup; DuckDB is a single binary with zero infrastructure
DuckDB	Zero-config S3 support, Parquet-native columnar streaming, single binary, SQL interface

Key differences from raw boto3 benchmarks: (1) **Columnar access** -- analytics queries read specific columns, not entire files. (2) **Predicate pushdown** -- WHERE clauses skip irrelevant row groups. (3) **Compression** -- Parquet transfers compressed bytes. (4) **Multipart uploads** -- DuckDB uses S3 multipart protocol differently than boto3.

Test Data Schema

Parquet test files generated via generate_data_parquet.py with 5 columns per file:

Column	Type	Purpose
id	INT	Sequential identifier
payload	VARCHAR ~1KB	Pads file to target size
timestamp	TIMESTAMP	Synthetic timestamps
category	INT (0-9)	Filtering and GROUP BY
value	DOUBLE	Random values for aggregation

Size Tier	Files	Per-File Size
1 MB	100	~1 MB Parquet
16 MB	100	~16 MB Parquet
32 MB	100	~32 MB Parquet

Workload Modes

write-only

For each file: (1) Read local Parquet into DuckDB. (2) COPY TO s3://bucket/benchmark/file.parquet. (3) Measure wall-clock time. Warmup: 10% of files at target thread count before measurement begins.

read-only

Seed bucket first (metrics discarded), then for each file: SELECT SUM(value), COUNT(*), MAX(timestamp) FROM read_parquet(s3://bucket/file.parquet). Forces full columnar scan -- not just Parquet footer metadata.

mixed

Sequential: write phase followed by read phase. Both phases measured and reported independently.

heavy

Split threads in half: writers continuously COPY Parquet files to S3; readers continuously run column-materializing queries. Run for 30 seconds. Readers re-list available S3 keys every 5 seconds to discover fresh writes. Reports throughput and P50/P95/P99 latency for both operations.

Metrics Collected

Metric	Description
aggregate_throughput_mbps	Total data (MB) / wall-clock time (s)
latency_p50	Median per-file operation time
latency_p95	95th percentile latency
latency_p99	99th percentile latency
latency_mean	Average latency
latency_stddev	Standard deviation of latencies
success / failed	Operation success counts
write_success_rate	% of writes that succeeded (heavy mode)
read_success_rate	% of reads that succeeded (heavy mode)

Interpreting DuckDB Results

- DuckDB write throughput may be **lower** than boto3: Parquet encoding + S3 multipart upload overhead
- DuckDB read throughput may be **higher** for analytical queries: columnar projection reduces bytes read
- Latency patterns differ: DuckDB has higher per-operation overhead (SQL parsing, Parquet decode) but lower total bytes transferred for selective reads
- Heavy mode shows real contention: how well the storage system handles concurrent mixed analytical queries

Appendix E -- DuckLake Benchmark Methodology

Architecture

DuckLake adds database-backed metadata (snapshots, time travel, ACID transactions) on top of Parquet files stored on S3. The architecture stacks four layers:

Layer	Component	Purpose
Query engine	DuckDB connection	SQL interface + columnar execution
S3 access	httpfs extension	Read/write Parquet files to S3
Lakehouse format	ducklake extension	Snapshot management, ACID, time-travel
Metadata store	SQLite metadata.db (local disk)	Snapshots, lineage, transaction log
Data store	Parquet files on S3	Actual row data

Why SQLite metadata (not on S3)? All three targets use the same local SQLite instance -- the only variable between targets is S3 storage speed. This is a deliberate isolation design for fair comparison.

Table Schema

```
CREATE TABLE my_db.events (id UUID, ts TIMESTAMP, sensor_id INTEGER, value DOUBLE, label VARCHAR)
```

Realistic IoT/sensor event pattern -- common in streaming data lake workloads.

Data Generation

Unlike the DuckDB benchmark (pre-generated local files), DuckLake generates data inline via SQL INSERT in batches of 1,000 rows. Actual S3 Parquet size measured via boto3 list_objects_v2 after all writes complete -- not estimated from row counts.

Size Tier	Rows	Approximate S3 Data
1 MB	5,000	~0.6 MB
16 MB	80,000	~10 MB
32 MB	160,000	~20 MB

All 8 Benchmark Modes

Mode	What It Tests	Key Detail
write-only	Bulk INSERT throughput through full DuckLake pipeline	SQL -> Parquet encoding -> S3 upload -> SQLite metadata update (3 steps vs 1 for raw COPY)
read-only	Analytical query latency against seeded lakehouse tables	Seed is untimed; 5 analytical GROUP BY / aggregation queries measured
mixed	Sequential write then read (both measured)	Isolates write-then-read pipeline latency
heavy	Concurrent reads + writes for 30 seconds	Half threads write, half read; tests real S3 + SQLite WAL contention
transactions	200 multi-table ACID transactions	BEGIN -> INSERT orders + 1-5 items -> COMMIT; tests DuckLake ACID guarantees end-to-end
time-travel	Snapshot creation + historical queries	CREATE SNAPSHOT; SELECT COUNT(*) FROM my_db.events AT VERSION n
snapshots	10 snapshot creates + list all	Isolates SQLite metadata performance for version management
concurrent	Multiple independent DuckLake writers	Per-worker SQLite; tests S3 throughput NOT SQLite WAL bottleneck

[i] Note on DuckLake read metrics: reads report rows/sec and latency, NOT throughput_mbps. Each analytical query scans only a fraction of total Parquet data via column pruning. Attributing total file size to every query would inflate throughput artificially.

Why concurrent Mode Uses Per-Worker Metadata

The original shared-metadata approach measured SQLite single-writer bottleneck (13% success rate), not S3 storage performance. With per-worker SQLite metadata files, write success is 100% and the benchmark correctly measures S3 throughput under concurrent DuckLake INSERT workloads.

Run Isolation and Cleanup

Each DuckLake run is fully isolated to prevent cross-run contamination:

- Creates a fresh S3 bucket per run: benchduck-{target}-run{id}-{timestamp}
- Creates local metadata_*.db files (one per adapter instance) on first connection
- After completion: removes all metadata DBs and empties the S3 bucket
- Next run starts with clean state: empty bucket, no cached metadata, no stale snapshots

Interpreting DuckLake Results

Question	Answered By
Can this S3 system handle a real data lake?	DuckLake benchmark all modes
What is the overhead of ACID guarantees?	DuckLake transactions mode -- compare P50 vs raw DuckDB writes
How fast can I query historical data?	DuckLake time-travel mode -- AT VERSION n query latency
What happens with concurrent lakehouse writers?	DuckLake concurrent mode -- per-worker metadata isolates S3 throughput
Is DuckLake metadata overhead significant?	Compare DuckLake vs DuckDB write throughput at same file size
Does storage backend affect transaction reliability?	transactions mode -- all three targets achieve 100% ACID success

[i] DuckLake writes are slower than raw DuckDB COPY: every INSERT requires Parquet encoding + S3 upload + SQLite metadata update (3 steps vs 1). Time travel is cheap: snapshot creation is SQLite metadata only; AT VERSION queries add minimal overhead over a regular read.

Appendix F -- Iceberg Benchmark Methodology

How Iceberg Metadata Works on S3

Apache Iceberg is the industry-standard open table format. Unlike DuckLake (SQLite metadata), Iceberg stores all metadata as files in the table directory on S3. Every write creates a new snapshot with its own metadata files:

File	Role
data/00000-1-<uuid>.parquet	Data file written by each commit
metadata/v1.metadata.json	Table schema, partition spec, snapshot list
metadata/v2.metadata.json	New snapshot after each write operation
metadata/snap-<uuid>.avro	Manifest list -- which data files are in this snapshot
metadata/<uuid>.avro	Manifest file -- per-file column stats
metadata/version-hint.txt	Pointer to latest metadata version number

This enables: **time travel** (query at any previous snapshot), **ACID transactions** (each snapshot is atomic), **schema evolution** (add/rename columns without rewriting data), and **partition evolution** (change partitioning without rewriting existing data).

Why DuckDB for Iceberg Simulation (Not Pylceberg)?

Approach	Decision
Pylceberg + S3	PyArrow multipart upload fails on MinIO-compatible endpoints -- known compatibility issue
DuckDB + https	Same engine as DuckDB/DuckLake benchmarks -- consistent S3 access, no extra dependencies
Iceberg simulation	Writes multiple small Parquet files (like Iceberg data files), uses glob() for manifest-style listing

Table Schema

Column	Type	Description
id	INTEGER	Sequential identifier
category	VARCHAR (A-D)	Category label for grouping
value	DOUBLE	Random float for aggregation
event_time	TIMESTAMP	Partition key (day transform)

Data Generation

Data generated inline using DuckDB generate_series() -- no local files required. Each batch writes one Parquet file to S3 as: data/events/000001-<uuid>.parquet (mirroring Iceberg data file naming convention).

Size Tier	Rows
1 MB	5,000
16 MB	80,000
32 MB	160,000

All 9 Benchmark Modes

Mode	What It Tests	Key Detail
write-only	Parquet file creation throughput	Each batch = one Parquet file on S3; measures Parquet encode + S3 PUT latency
read-only	Analytical query latency on individual data files	5 queries: COUNT, GROUP BY, column stats, range filter, HAVING aggregation
mixed	Sequential write then read	Isolates write-then-read pipeline
heavy	Concurrent writers + readers for 30 seconds	Half threads write new files; half read random existing files
time-travel	Historical file reads	Write batches to distinct paths; read each by path (simulates snapshot-per-commit)
snapshots	File listing overhead	10 Parquet files created; glob(s3://bucket/data/*.parquet) tests LIST latency
change-detection	Diff two file sets	Read two snapshot paths as DataFrames; diff on primary key

Mode	What It Tests	Key Detail
update	New file simulating row update	COPY new Parquet to data/update_<ts>.parquet; measures create-new-file pattern
delete	New file simulating row delete	COPY new Parquet to data/delete_<ts>.parquet; measures delete-marker pattern

Iceberg vs DuckLake Comparison

Dimension	Iceberg	DuckLake
Metadata location	S3 files (network round-trip per operation)	Local SQLite (sub-millisecond)
Time travel	Snapshot file listing + path-based queries	AT VERSION n (SQLite version lookup)
Update/Delete	New Parquet file per operation (copy-on-write)	SQL INSERT (DuckLake manages Parquet)
Snapshot listing	glob() over S3 -- slower but no database dependency	SQLite query -- fast but requires catalog
Portability	Self-contained on S3; no external service needed	Requires SQLite file alongside data

Metrics Collected

Metric	Description
aggregate_throughput_mbps	Total data (MB) / wall-clock time (s)
latency_p50 / p95 / p99	Per-operation latency percentiles
latency_mean / latency_stdev	Average and variance
success / failed	Operation success counts
write_success_rate / read_success_rate	Success rates in heavy mode
rows_returned	Rows returned by analytical queries
snapshot_count	Number of snapshot files created
change_detection	Inserts/deletes/updates between snapshot diffs

What Iceberg Tests That Other Engines Do Not

Question	Answered By
Can this S3 system handle multiple concurrent small Parquet file writes?	All Iceberg modes -- each batch = one file, many batches run in parallel
How fast can this system create and list snapshot files?	snapshots mode -- 10 Parquet files + glob() listing latency
What is the cost of reading historical data from S3-only metadata?	time-travel mode -- path-based historical queries, no catalog
Can this system handle concurrent Parquet writers without coordination?	heavy mode -- half threads write new files, half read random files
How fast can this system detect changes between two file versions?	change-detection mode -- read two snapshot paths, diff on primary key
How does file-based metadata (Iceberg) compare to DB-based (DuckLake)?	Cross-engine table in Section 8.4 -- same S3 backend, different metadata planes

Interpreting Iceberg Results

- **Iceberg writes slower than DuckDB COPY at same size?** Expected -- Iceberg simulation generates data in-process via `generate_series()` which adds CPU overhead vs reading a pre-built local Parquet file
- **Iceberg reads faster than DuckDB reads?** Likely smaller effective dataset -- Iceberg benchmark uses synthetic data with fewer distinct values; predicate pushdown is more effective
- **High change-detection variance?** S3 LIST latency is non-deterministic; glob() over many files has higher variance than a single HEAD request
- **Heavy mode write P50 >> sequential write P50?** Same quorum/GC pattern as DuckDB heavy mode -- confirms the bottleneck is in the storage backend, not the Iceberg simulation layer

-> The Iceberg benchmark answers: How well does this S3 storage system support Iceberg-style file-based data patterns -- multiple small Parquet files, manifest listing, snapshot reads, and concurrent writes?